

Reviewer Pack — Minimal Order of Quasi-random Sequences and Communication Co...

Entry 0f2db83b086f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 04:38:03 UTC

Minimal Order of Quasi-random Sequences and Communication Complexity

Entry ID: 0f2db83b086f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 04:38:03 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Number Theory (Quasi-random Sequences) **Field B** (complexity object): Communication Complexity

Statement:

For all boolean functions f with n inputs, the number of quasi-random sequences required to uniformly cover its communication complexity class $C(f)$ is bounded by $O(\log^2 n)$. Equivalently, $|C(f)| = \Theta(\log^2 n)$.

Rationale (proposer's reasoning):

Quasi-random sequences have been used in coding theory and cryptography but are rarely applied to communication complexity. If such sequences can provide a logarithmic upper bound on the communication complexity class, it may expose a new perspective on understanding the limits of efficient communication.

Taxonomy category: communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4e0237b47becafbd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for a statistically significant majority (≥ 0.8) of generated boolean functions, the number of quasi-random sequences required to cover all possible outputs of f is within $O(\log^2 n)$, with a mean difference from $\log^2 n$ less than or equal to 3 across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - combinatorial number theory AND quasi-random sequences AND communication complexity - quasi-random sequence AND order AND communication complexity class - communication complexity AND boolean function AND quasi-random sequence coverage

Top relevant hits considered: - [http://arxiv.org/abs/math/0501315v2] Taming the wild in impartial combinatorial games - [http://arxiv.org/abs/1005.2692v2] An Extreme Family of Generalized Frobenius Numbers - [http://arxiv.org/abs/2010.10175v2] Primitive divisors of sequences associated to elliptic curves with complex multiplication - [http://arxiv.org/abs/1005.2254v9] On Universal Complexity Measures - [http://arxiv.org/abs/1804.04431v2] Ordered Sequence Detection and Barrier Signal Design for Digital Pulse Interval Modulation in Optical Wireless Communication - [http://arxiv.org/abs/1910.13723v1] On the maximum order complexity of the Thue-Morse and Rudin-Shapiro sequence - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity(f):
        n = len(f)
        cc = 0
        for i in range(2**n):
            input_bits = [int(b) for b in format(i, f'0{n}b')]
            output = f[input_bits]
            if output not in [0, 1]:
                continue
            cc += 1
        return cc

    def quasi_random_sequences(n):
        sequences = []
        for _ in range(2**n):
            sequence = [random.choice([0, 1]) for _ in range(n)]
            sequences.append(sequence)
```

```

    return sequences

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        f = [random.choice([0, 1]) for _ in range(2**n)]
        cc = communication_complexity(f)
        if cc == 0:
            continue
        instances_tested += 1
        n_max = max(n_max, n)
        expected_log_cc = math.log2(cc) ** 2
        actual_log_cc = math.log2(instances_tested)
        total_metric_value += abs(expected_log_cc - actual_log_cc)
        if abs(expected_log_cc - actual_log_cc) > 3:
            conjecture_holds = False
            counterexample = f"n={n}, cc={cc}, instances_tested={instances_tested}"

return {
    "metric_name": "log_difference",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']}\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91f4dd60.py", line 76, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91f4dd60.py", line 49, in run_trial
    cc = communication_complexity(f)
        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91f4dd60.py", line 26, in communication
    output = f[input_bits]
            ~~~~~
```

TypeError: list indices must be integers or slices, not list

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the error in the test code to allow for proper testing of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19773
2	preregistration	ollama_remote	glm4:latest	0	0	9678
3	novelty	ollama_remote	glm4:latest	0	0	8309
4	novelty	ollama_remote	glm4:latest	0	0	10382
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43447
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8948
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9320
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8438
9	judge	ollama_remote	glm4:latest	0	0	15202

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133496 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0f2db83b086f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f2db83b086f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0f2db83b086f.tar.gz` (if generated)